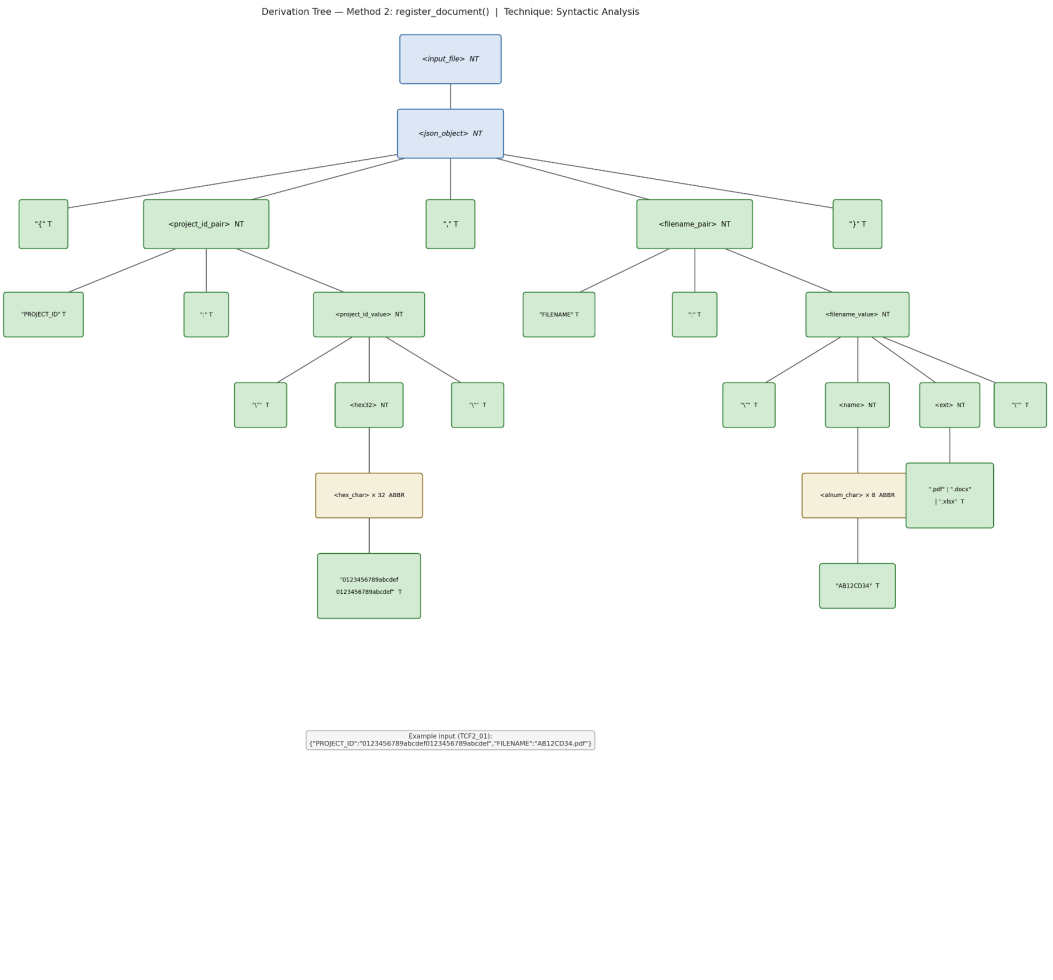
Method 2: register_document

CONTENTS

Derivation Tree & Grammar
Groups of Test Cases
Control Flow Graph
Definition of Basic Paths
Test Case Definitions for Control Flow

Method 2: check_project_budget

Derivation Tree



Grammar

```
<input_file>       ::= <json_object>
<json_object>      ::= "{" <project_id_pair> "," <filename_pair> "}"
<project_id_pair>  ::= '"PROJECT_ID"' ":" <project_id_value>
<filename_pair>    ::= '"FILENAME"' ":" <filename_value>
<project_id_value> ::= '"' <hex32> '"'
<hex32>            ::= <hex_char>{32}
<hex_char>         ::= [0-9] | [a-f]
<filename_value>   ::= '"' <name> <extension> '"'
<name>             ::= <alnum_char>{8}
<alnum_char>       ::= [a-z] | [A-Z] | [0-9]
<extension>        ::= ".pdf" | ".docx" | ".xlsx"
```

Groups of Test Cases

Valid Inputs

These tests verify that the method accepts correctly structured JSON files and valid field values.

- **TCF2_01**: valid `PROJECT_ID` and valid filename ending in `.pdf`
- **TCF2_02**: valid `PROJECT_ID` and valid filename ending in `.docx`
- **TCF2_03**: valid `PROJECT_ID` and valid filename ending in `.xlsx`
- **TCF2_29**: valid filename where the 8-character name is composed only of digits, confirming that digits are allowed in `<NAME>`

These cases confirm the positive behavior required by:

- **CM-FR-02-O1**: return SHA-256 signature
- **CM-FR-02-O2**: append data to `all_documents.json`

Input File Access Error

- **TCF2_04** checks the case where the input path does not exist.

This verifies the exception described in:

- **CM-FR-02-O3: input file not found**

JSON Syntax Errors

These test cases verify that the method rejects files that are not valid JSON documents.

- **TCF2_05**: missing opening brace
- **TCF2_06**: missing closing brace
- **TCF2_07**: missing comma between fields
- **TCF2_08**: extra comma before closing brace

These tests are important because they validate the parser behavior before any business validation is applied. They correspond to:

- **CM-FR-02-O3: file is not JSON formatted**

Wrong JSON Structure

These cases are syntactically valid JSON, but they do not follow the expected grammar of the input object.

- **TCF2_09**: top-level JSON is an array instead of an object
- **TCF2_10**: missing `PROJECT_ID`
- **TCF2_11**: missing `FILENAME`

- **TCF2_12**: duplicated `PROJECT_ID`
- **TCF2_13**: duplicated `FILENAME`
- **TCF2_26**: misspelled key name `PROJECTID`
- **TCF2_27**: empty JSON object

These tests verify:

- the root must be a single object
- both required keys must appear exactly once
- key names must match the specification exactly

They correspond to:

- **CM-FR-02-O3: JSON does not have the expected structure**

Invalid PROJECT_ID Values

These tests verify semantic validation of the `PROJECT_ID` field.

- **TCF2_14**: `PROJECT_ID` is not a string
- **TCF2_15**: `PROJECT_ID` has 31 hexadecimal characters
- **TCF2_16**: `PROJECT_ID` has 33 hexadecimal characters
- **TCF2_17**: `PROJECT_ID` contains a non-hexadecimal character

These cases confirm that `PROJECT_ID` must be:

- a string
- exactly 32 hexadecimal characters

They correspond to:

- **CM-FR-02-O3: JSON data has no valid values**

Invalid FILENAME Values

These tests verify semantic validation of the `FILENAME` field.

Type validation

- **TCF2_18**: `FILENAME` is not a string

Base name length validation

- **TCF2_19**: base name has 7 characters
- **TCF2_20**: base name has 9 characters

Base name character set validation

- **TCF2_21**: base name contains underscore

- **TCF2_22**: base name contains a space

Extension validation

- **TCF2_23**: no extension
- **TCF2_24**: disallowed extension `.txt`
- **TCF2_25**: wrong case extension `.PDF`

These tests verify the rule that `FILENAME` must follow the form:

```
<NAME><EXTENSION>
```

where:

- `<NAME>` contains exactly 8 alphanumeric characters
- `<EXTENSION>` is one of `.pdf`, `.docx`, `.xlsx`

3.7 Internal Processing Error

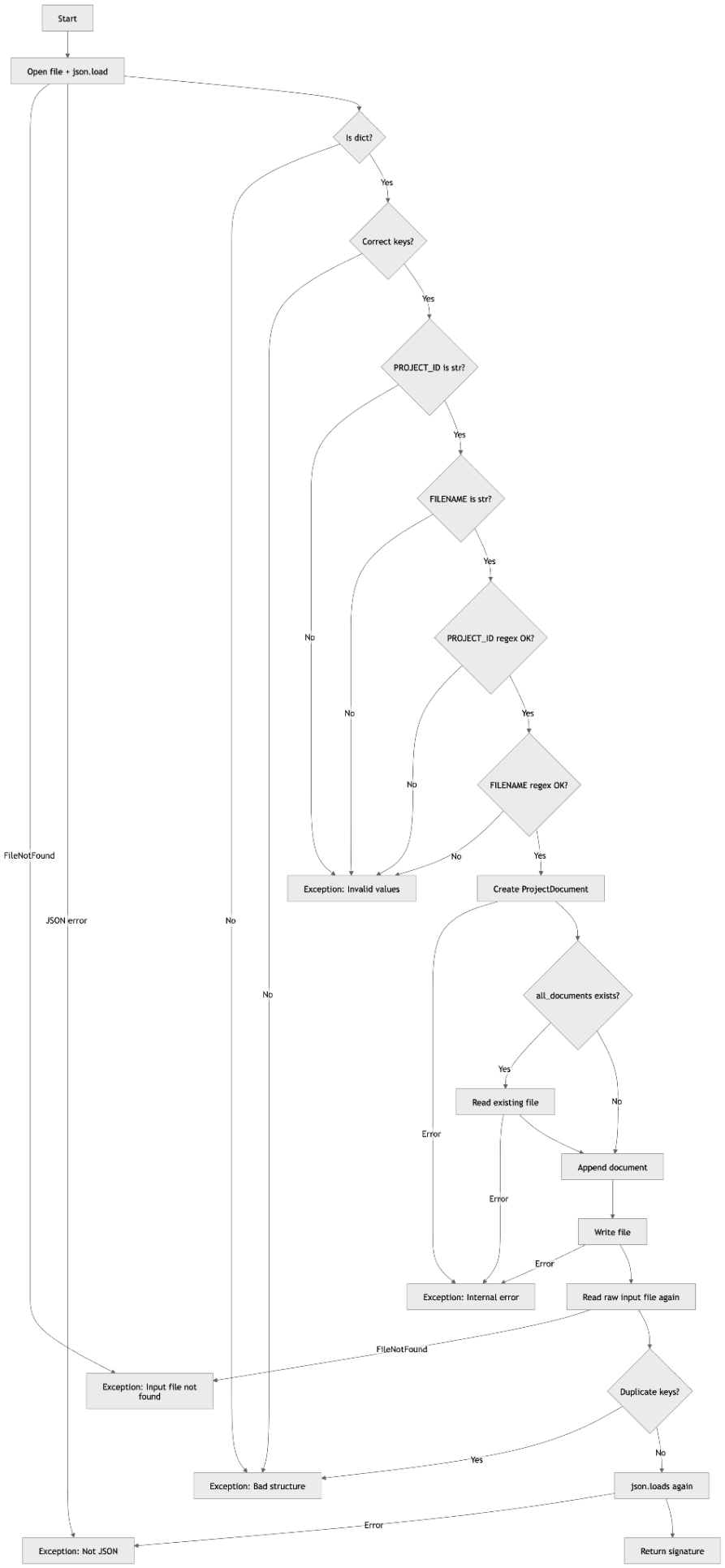
- **TCF2_28** verifies that the method raises an exception if a failure occurs during SHA-256 signature generation.

This test does not validate input syntax or input values, but rather the robustness of internal processing. It corresponds to:

- **CM-FR-02-O3: internal processing error when getting the file_signature**

Control Flow Graph

1 Start
2 `open(input_file)` + `json.load`
3 first `FileNotFoundError`
4 first `JSONDecodeError`
5 `isinstance(data, dict)?`
6 bad structure
7 exact keys?
8 bad structure
9 `PROJECT_ID` is `str`?
10 invalid values
11 `FILENAME` is `str`?
12 invalid values
13 `PROJECT_ID` regex valid?
14 invalid values
15 `FILENAME` regex valid?
16 invalid values
17 create `ProjectDocument` / get signature / `to_json`
18 internal processing error
19 `all_documents.json` exists?
20 read existing `all_documents.json`
21 internal processing error
22 append document
23 write `all_documents.json`
24 internal processing error
25 second `open(input_file)` + `read`
26 second `FileNotFoundError`
27 duplicate keys?
28 bad structure
29 second `json.loads(content)`
30 second `JSONDecodeError`
31 return signature


Start
Open file + json.load
Is dict?
Yes
Correct keys?
Yes
PROJECT_ID is str?
Yes
FILENAME is str?
Yes
PROJECT_ID regex OK?
Yes
FILENAME regex OK?
Yes
Create ProjectDocument
all_documents exists?
Yes
Read existing file
No
Append document
Write file
Read raw input file again
Duplicate keys?
No
json.loads again
Return signature
No
No
No
No
No
No
Exception: Invalid values
FileNotFound
Exception: Input file not found
JSON error
Exception: Not JSON
Error
Error
Error
Error
Exception: Internal error
FileNotFound
Yes
Exception: Bad structure

Definition of Basic Paths

The following basic paths have been identified from the control flow graph:

Successful Paths

- **BP1 (Baseline path)**: Valid input, `all_documents.json` does not exist → document is registered successfully.
- **BP2**: Valid input, `all_documents.json` exists and is correctly read → document is appended and registered successfully.

Input and Parsing Error Paths

- **BP3**: Input file not found during the first read.
- **BP4**: Input file exists but is not JSON formatted.

Structure Validation Paths

- **BP5**: JSON root is not a dictionary.
- **BP6**: JSON does not contain the expected keys.

Data Validation Paths

- **BP7**: `PROJECT_ID` is not a string.
- **BP8**: `FILENAME` is not a string.
- **BP9**: `PROJECT_ID` does not match the required 32-hexadecimal format.
- **BP10**: `FILENAME` does not match the required pattern.

These paths correspond to the **main independent execution paths** needed to achieve decision and branch coverage.

Additional Paths of the Control Flow

In addition to the basic paths, the method includes **exception and edge-case paths** that must be tested to ensure robustness:

- **AP1**: Internal error when creating the `ProjectDocument` or generating the signature.
- **AP2**: Failure when reading an existing `all_documents.json` file (corrupted or inaccessible).
- **AP3**: Failure when writing to `all_documents.json`.
- **AP4**: Input file not found during the second read.
- **AP5**: Duplicate keys (`PROJECT_ID` or `FILENAME`) detected in the raw JSON content.
- **AP6**: Failure during the second JSON parsing.

These paths are important because they represent **exception handling branches**, which are part of the control flow and must be tested in structural testing.

Test Case Definitions for Control Flow

Each identified path has been mapped to at least one test case to ensure full coverage.

Successful Test Cases

- **ST-01 (BP1)**: Valid input file, `all_documents.json` does not exist → expected result: signature returned.
- **ST-02 (BP2)**: Valid input file, `all_documents.json` exists → expected result: signature returned.

Input and Parsing Error Test Cases

- **ST-03 (BP3)**: Input file does not exist → expected result: `Input file not found` exception.
- **ST-04 (BP4)**: Input file not JSON formatted → expected result: `File is not JSON formatted` exception.

Structure Validation Test Cases

- **ST-05 (BP5)**: JSON root is not a dictionary → expected result: `JSON does not have expected structure`.
- **ST-06 (BP6)**: JSON keys do not match expected set → expected result: `JSON does not have expected structure`.

Data Validation Test Cases

- **ST-07 (BP7)**: `PROJECT_ID` not string → expected result: `JSON data has no valid values`.
- **ST-08 (BP8)**: `FILENAME` not string → expected result: `JSON data has no valid values`.
- **ST-09 (BP9)**: Invalid `PROJECT_ID` format → expected result: `JSON data has no valid values`.
- **ST-10 (BP10)**: Invalid `FILENAME` format → expected result: `JSON data has no valid values`.

Additional Control Flow Test Cases

- **ST-11 (AP1)**: Internal error during document creation/signature → expected result: internal processing exception.
- **ST-12 (AP2)**: Corrupted `all_documents.json` → expected result: internal processing exception.

- **ST-13 (AP3)**: Write failure to `all_documents.json` → expected result: internal processing exception.
- **ST-14 (AP4)**: Input file missing during second read → expected result: `Input file not found`.
- **ST-15 (AP5)**: Duplicate keys in JSON → expected result: `JSON does not have expected structure`.
- **ST-16 (AP6)**: Failure in second JSON parsing → expected result: `File is not JSON formatted`.